ENGINEERING CHANGE NOTE (ECN)

PROJECT: PROJECT-CE

ORIGINATION: Prof. Kidd, Instructor, UMBC

PURPOSE OF CHANGE: Custom millisecond timer

DOCUMENT NUMBER: ECN1.2

DATE OF PUBLICATION: 10/15/2026

LEAD ENGINEER: Kid Kidd, UMBC

TEAM LEAD: Nyi Thu Kha, UMBC

EXECUTIVE SUMMARY:

This ECN documents the change from the Project 2 (PROJECT-CE) design to a custom millisecond timer implementation. The customer requirements, high-level technical requirements, hardware configuration, and cyclic executive architecture are unchanged. The Arduino millis() function has been replaced by a Developer-owned mymilli() function backed by a custom Timer0 CTC ISR and locally defined register constants. Testing requirements have been extended to verify the new timer. The formal design document has been revised accordingly.

HIGH-LEVEL DESCRIPTION:

Replace Arduino's millis() function in PROJECT-CE with a different mymilli() function, including a custom Timer0 ISR and custom register definitions, without changing the customer or high-level technical requirements.

REASON FOR CHANGE:

Results in (1) a more flexible design allowing the design team to customize and extend the function, (2) a design entirely under company control isolated from any future changes in the third-party arduino.h, and (3) avoidance of possible future IP ownership claims.

Changed Documents:

10/15/2026 (CMPE311-PROJECT-CE-SDD1.1 → CMPE-PROJECT-MYMILLI-SDD.1.2)
System Design Document:

- **Changed Document Name:** “"Cyclic Executive Task Manager – System Design Document" to "Custom Millisecond Timer – System Design Document".
- **Changed Document Reference:** "CMPE 311-PROJECT-CE-SDD.1.1" to "CMPE 311-PROJECT-MYMILLI-SDD.1.2".
- **Changed Date of Publication:** 9/29/2026 to 10/15/2026

- **HIGH-LEVEL DESCRIPTION:**
  - **Added:** “and a custom millisecond timer” at the end.
- **DESCRIPTION:**
  - **Added:** "The Arduino millis() function has been replaced by a custom mymilli() function backed by a custom Timer0 CTC ISR and locally defined register constants." after the cyclic executive description.
- **RESULTS SUMMARY:**
  - **Added:** "and the custom timer" before "were satisfied".
- **REFERENCE:**
  - **Added:** "[1] Nyi Thu Kha, Cyclic Executive Task Manager — System Design Document, CMPE311-PROJECT-CE-SDD.1.1" as first reference. The previous Project 1 reference is renumbered to [2].
- **DEFINITIONS:**
  - **Added:** ISR, CTC Mode, Prescaler, Atomic Access, mymilli().
- **ACRONYSMS AND ABBREVIATIONS:**
  - **Added:** CTC, PWM.
- **EDUCATIONAL CONTRAINTS:**
  - **Added:** EC.7, EC.7.1, EC.7.2, EC.7.3, EC.7.3.1, EC.7.4, EC.7.5.
- **ADDITIONAL DESIGN CONSIDERATIONS:**
  - **Added:** Items 10-14. Item 10 notes that Arduino millis() is replaced by mymilli(). Item 11 describes the Timer0 CTC configuration with OCR0A = 249 for exactly 1 ms per interrupt. Item 12 lists the Developer-defined register constants. Item 13 describes the atomic read using cli() and SREG restore. Item 14 notes that Arduino millis() and delay() no longer function correctly and are not used.
- **SYSTEM DESIGN:**
  - **Added:** New subsection "Custom Millisecond Timer" under SYSTEM DESIGN. Contains description of the Timer0 CTC configuration, the ISR, the atomic read, custom register constants, and extern "C" linkage guards.
- **TESTING AND VALIDATION REQUIREMENTS:**
  - **Added:** T.18, T.19, T.20, T.21, T.22.
  - **Changed:** Section now references [1] for T.0 – T.17 rather than restating them.
  - **Changed:** Table 4 now summarizes T.0 – T.17 as a range referencing [1] and lists T.18 – T.22 individually.

- **TESTING RESULTS**
    - **Added:** Rows for T.23 – 2.27.
    - **Changed:** GitHub repository link updated from Project 2 to Project 3.

Added Documents:

- ECN1_2 – This Engineering Change Note, dated 10/15/2026.
- mymilli.h – Custom timer header, CMPE311-PROJECT-MYMILLI-HDR.1.0, dated 10/15/2026.
- mymilli.c – Custom timer implementation, CMPE311-PROJECT-MYMILLI-SRC.1.0, dated 10/15/2026.

Removed Documents:

None.

Replaced Documents:

None.

Any Additional Design Considerations:

1. Timer0 is configured in CTC mode with a prescaler of 64 and OCR0A = 249, producing exactly one compare-match interrupt per millisecond.
2. The custom ISR (TIMER0_COMPA_vect) increments a 32-bit volatile counter.
3. mymilli() reads the counter atomically using cli() and SREG restore.
4. Register access uses Developer-defined constants (TIMER0_CTRL_A, TIMER0_CTRL_B, TIMER0_COUNT, TIMER0_COMPARE_A, TIMER0_INT_MASK) instead of avr.h names.
5. The header wraps function declarations in extern "C" guards to bridge the C++ (.ino) and C (.c) compilation units.
6. No third-party multitasking libraries or RTOS are used so EC.5 remains satisfied.
7. All Project 1 and Project 2 customer requirements, high-level technical requirements, educational constraints, hardware, and user-visible behavior are unchanged.